

Glossar zu Begriffen

Microkernel

Ein Microkernel beschreibt eine Architektur, bei der sich im Kernel nur Basisfunktionalitäten befinden. Nicht essenzielle Services laufen in Prozessen die auf den Kernel zugreifen und kommunizieren über den Kernel miteinander.

Services die ein Microkernel bereitstellt sind:

- Process Switching
- Simple Memory Management
- Interrupts
- I/O
- wie schon erwähnt Nachrichtenaustausch (und Kontrolle)

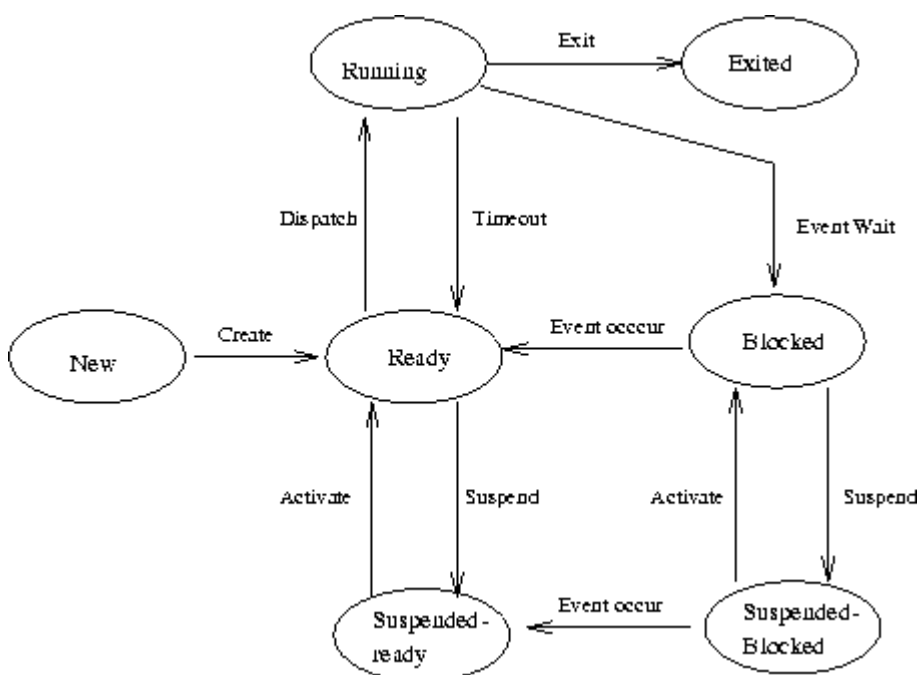
Vorteile:

- Kernel bleibt klein
- sehr flexibel, weil nur nötige Funktionalitäten angehängt werden
- Interfaces bleiben einheitlich
- portabel und verteilbar

Nachteile:

- Betriebssystemroutinen können langsamer sein, weil sie als Prozesse realisiert sind.

Prozesszustände



Process Switch

Kontextwechsel oder Taskswitching (englisch context switch) nennt man den Vorgang in einem Betriebssystem, bei dem die Bearbeitung des aktuellen Prozesses (oder auch Threads, Tasks oder Programms) unterbrochen wird (z. B. nach einer festgelegten Zeitspanne durch einen Timer-Interrupt oder bei Systemaufrufen) und zu einer anderen Routine gewechselt wird. Ein Process Switch kann immer dann passieren, wenn das BS im Besitz der CPU ist:

- Supervisor Call: expliziter Aufruf durch das Programm (I/O, ...)
- Trap: bedingt durch aktuelle Instruktion (z.B. Auftreten eines Fehlers)
- Interrupt: Ursache liegt außerhalb des Prozesses, Kontrolle geht an Interrupt Handler und BS

Process und Mode Switch

Der Mode Switch ist die Basis eines jeden Process Switch. Ein Process Switch kann nur passieren wenn eine Mode Switch passiert, aber nicht jeder Mode-Switch führt zu einem Process Switch. Der Mode Switch wechselt zwischen User und Kernel Mode.

Siehe Frage zu Process Switch um Ereignisse zu sehen.

Ready und Blocked

- Ready
 - Prozess ist bereit zur Ausführung (wartet nur auf Zuteilung der CPU)
- Blocked
 - Prozess wartet auf ein Ereignis (z.B. Beendigung von I/O), ist nicht lafbereit

Kernel Level vs. User Level Threads

- KLT: Blockieren auf Thread-Basis
- ULT: Blockieren auf Prozessbasis

Process Control Block

Der PCB ist Teil des Process Image, welches im Virtual Memory ist und beinhaltet die Process ID, Process Control Information und Process State Information.

Process ID besteht aus der eigenen Prozessnummer (Index im Process Table), der User ID und der Prozessnummer des erstellenden Prozesses

Process State Information besteht aus den Registerinhalten:

- Kontroll- und Statusregister: Befehlszähler und PSW(P.S. Word - Modusinformation, Statusbits)
- Stack Pointer

Process Control Information dient der Scheduling und Zustandsinformation (wobei der Zustand, die

Priorität und das Event auf das gewartet wird gespeichert wird) und für Querverweise zu anderen Prozessen (Parent, Child, für Process Queues). Ebenso der IPC, Privilegien, Memory Management und Ressourcenallokation.

Process Image

Das Process Image befindet sich im V-Mem und speichert:

- User Program
- User Data
 - modifizierbarer Bereich des User Space (Daten, User Stack, modifizierbare Programme)
- System Stack
 - Parameter und Calling Addr. von System Calls
- Process Control Block (Execution Context)
 - Process identification, processor state information, process control information

Prozesse und Threads

Prozesse können mehrere Threads beinhalten, wobei Prozesse für die Ressourcenverwaltung und Threads für das Dispatching (kurzfristiges Scheduling) verantwortlich sind.

Die Thread-Erzeugung, Thread-Zerstörung und Thread-Switch ist schneller. Threads können miteinander kommunizieren ohne den Kernel einzuschalten, aber dafür muss die Synchronisation selbständig gemacht werden.

Deadlock Avoidance

Deadlock Avoidance verhindert Reservierungen von Ressourcen, die zu einem Deadlock führen könnten. Dafür müssen die Anforderungen eines Prozesses im Vornherein bekannt sein und die Prozesse müssen sonst unabhängig voneinander sein. Die zwei Strategien, die es dafür gibt, sind Process Initiation Denial und Ressource Allocation Denial:

Process Initiation Denial startet einen Prozess nur dann, wenn seine Anforderungen zu keinem Deadlock führen können. Dieser Ansatz geht dadurch aber davon aus, dass alle Prozesse gleichzeitig gestartet werden und behindert die Parallelität.

Ressource Allocation Denial hat diese Einschränkung nicht, hier werden lediglich Unsafe States von Safe States unterschieden. Die Prozesse können nach und nach Ressourcen verlangen und bei jeder Allokation wird gecheckt ob es zu einem Safe State führt oder nicht. Nur "safe" Requests werden gewährt.

Deadlock Bedingungen

1. Mutual Exclusion: Nur ein Prozess darf auf eine Ressource zugreifen
2. Hold and Wait: Prozesse halten ihre Ressourcen und können auf andere warten
3. No Preemption: Prozesse können nicht unterbrochen werden bzw. deren Ressourcen können

nicht abgenommen werden.

4. Circular Wait: zyklische Ressourcenabhängigkeit zwischen Prozessen:

- Prozess A hat Ressource X und wartet auf Ressource Y und Prozess B hat Ressource Y und wartet auf Ressource X.

Kritischer Abschnitt

Drei Eigenschaften:

- Mutual Exclusion
- Progress (Entscheidung über nächsten Prozess der Eintreten darf soll erfolgen)
- Bounded Waiting (Anzahl der Eintritte werden beschränkt)

Bei Semaphores geschieht die Mutual Exclusion so:

```
P(S1);      P(S2);  
a();        a();  
V(S2);      V(S1);
```

Progress geschieht bei Semaphores über eine Queue -> FIFO Entscheidung welcher Prozess als nächstes eintreten darf.

Bounded Waiting geschieht über die Werte der Semaphoren.

Mutual Exclusion ohne Semaphoren

Ohne höhere Synchronisationskonstrukte wie Semaphoren ist es trotzdem noch möglich hardware- und softwareunterstützte Lösung zu nehmen.

Ein Beispiel für softwareunterstützte Lösungen sind Flags (globale Variablen) die immer wieder gecheckt werden (Busy Waiting), wie beim Deckeralgorithmus oder dem Petersen- Algorithmus.

Ein Beispiel für hardwareunterstützte Lösungen sind 1. Interrupt Disabling, wobei Prozesse durch einen Interrupt aus dem kritischen Bereich geholt werden, (v.a. bei Uniprozessorsystemen). Das BS wird dadurch aber beim Dispatchen eingeschränkt, deswegen gibt es auch Test and Set, welches auch auf Basis von Busy Waiting funktioniert.

Monitor

Der Monitor ist ein Softwaremodul, welches aus Prozeduren, lokalen Daten und einem Initialisierungscode besteht. Prozesse die auf den Monitor zugreifen wollen können dies nur über genannte Prozeduren machen. Ebenso kann maximal ein Prozess auf den Monitor und seine Daten zugreifen. Somit sorgt der Monitor für Mutual Exclusion und kann dafür auch Wartebedingungen haben. Die Wartebedingungen haben im Gegensatz zu Semaphoren keine Speichernde Wirkung, es wird ein Prozess ausgewählt wenn die Bedingung erfüllt wird.

CPU Scheduling

- First Come First Served (FCFS)Vergabe funktioniert so, dass die Prozesse nach Zeitpunkten der Ankunft abgearbeitet werden und bis zur Beendigung nicht unterbrochen. Das hat die Wirkung dass CPUintensive Prozesse und lange andauernde Prozesse stark begünstigt werden.
- Round RobinVergabe ist FCFS, aber es ist preemptive, d.h. es blockiert die Prozesse und die Anschließende Vergabe erfolgt zyklisch. Ein RR ist benachteiligend für IO-intensive Prozesse (werden von CPU intensiven Prozessen überholt) => deswegen:
- Virtual Round Robinhat eine Auxiliary Queue, welche eine höhere Priorität hat.
- Shortest Process Next (SPN)Vergabe geht immer an den als kürzesten geschätzten Prozess und der wird auch nicht unterbrochen. Die Response Time ist zwar besser als bei FCFS, aber es kann hier sogar zu Starvation kommen (bei langen Prozessen). Dadurch dass es non preemptive ist kann es auch zu Monopolisierung kommen.
- Shortest Remaining Time (SRT)Auswahl ist wie bei SPN, aber es ist preemptive. Die Service Times müssen dazu auch protokolliert werden und es kann trotzdem zu Starvation kommen
- Highest Response Ratio Nextberechnet Response Ratio welches aus Wartezeit+dauer / dauer besteht. Größtes RR wird gewählt. Es ist non preemptive und behandelt kurze Prozesse fair, wodurch es zu keiner Starvation kommt. Dafür muss aber die Service Time (Dauer) geschätzt werden.
- Feedback Scheduling.Priorität wird von bisheriger CPU-Time festgelegt (wenig ist besser). es ist preemptive. Es kann zu Starvation kommen, bei langen prozessen (Man kann aber auch wieder die Priorität erhöhen)

Relocation

ist die die Repositionierung von Code und Daten im Speicher und beschreibt das damit verbundene Virtual Memory Management. Weil beispielsweise nicht genügend Speicher vorhanden ist, muss er relocated werden. Prozesse belegen keine fixen Speicherstellen, wodurch man Referenzen speichern muss welche veraänderbar sind. Dadurch unterscheidet man zwischen logischen und physischen Adressen.

Starvation bei Scheduling

- FCFS
 - keine Starvation, weil die Prozesse sequentiell abgearbeitet werden. Jeder kommt mal dran
- Shortest Job First
 - Starvation von sehr langen Prozessen, wenn immer wieder neue kürzere Prozesse kommen
- Round Robin
 - keine Starvation, die Selection Function ist ident zu FCFS
- Priority Scheduling
 - Starvation ist ein großes Problem von Scheduling Algorithmen, die Prioritäten setzen, weil dadurch niedrig priorisierte Prozesse u.U. nicht mehr akzeptiert werden.

interne vs externe Fragmentierung

- Wird der Hauptspeicher in gleich grosse Einheiten (Frames) unterteilt, die nur als Ganzes vergeben werden, so tritt interne Fragmentierung auf. Als Interne Fragmentierung bezeichnet man den Effekt, dass bei einer Einteilung von Daten auf Speicherblöcke diese nur zum Teil gefüllt werden können.
- Ist kein genügend großes, zusammenhängendes Speicherstück mehr frei, obwohl die Summe der freien Stücke reichen würde, spricht man von externer Fragmentierung. Diese Fragmentierung kann durch Zusammenfügen (garbage collection) der freien Speicherstücke beseitigt werden.

Virtual Memory Management

- Dynamische Adressübersetzung
- Aufspaltung von Prozessen in Seiten (oder Segmente), die nicht hintereinander im Speicher abgelegt werden müssen
 - Prozesse müssen sich nicht als ganzes im Speicher befinden
 - nur "aktuell benötigte" Seiten im Speicher
 - mehr Prozesse passen in den Hauptspeicher
 - einzelne Prozesse können größer als der Hauptspeicher sein

Translation Lookaside Buffer

= Cache für Einträge der Seitentabelle

- Einträge enthalten Nummern der "zuletzt" verwendeten Seiten
- Assoziativer Zugriff beim Suchen
- Löschen des TLB bei jedem Context Switch

Virtual Memory Management Referenzierungen (TLB)

- TLB Miss ohne Page Fault: kann passieren, wenn ein miss im TLB passiert aber die Page trotzdem in der Page Table zu finden ist
- TLB Miss mit Page Fault: kann passieren, wenn sowohl ein Miss im TLB passiert als auch in der Page Table.
- TLB Hit ohne Page Fault: jeder Hit im TLB impliziert einen Hit in der Page Table, bei jedem TLB Hit gibt es keinen Page Fault
- TLB Hit mit Page Fault: kann unter normalen Umständen nicht passieren, weil im TLB nur Einträge der Pagetable sein können.

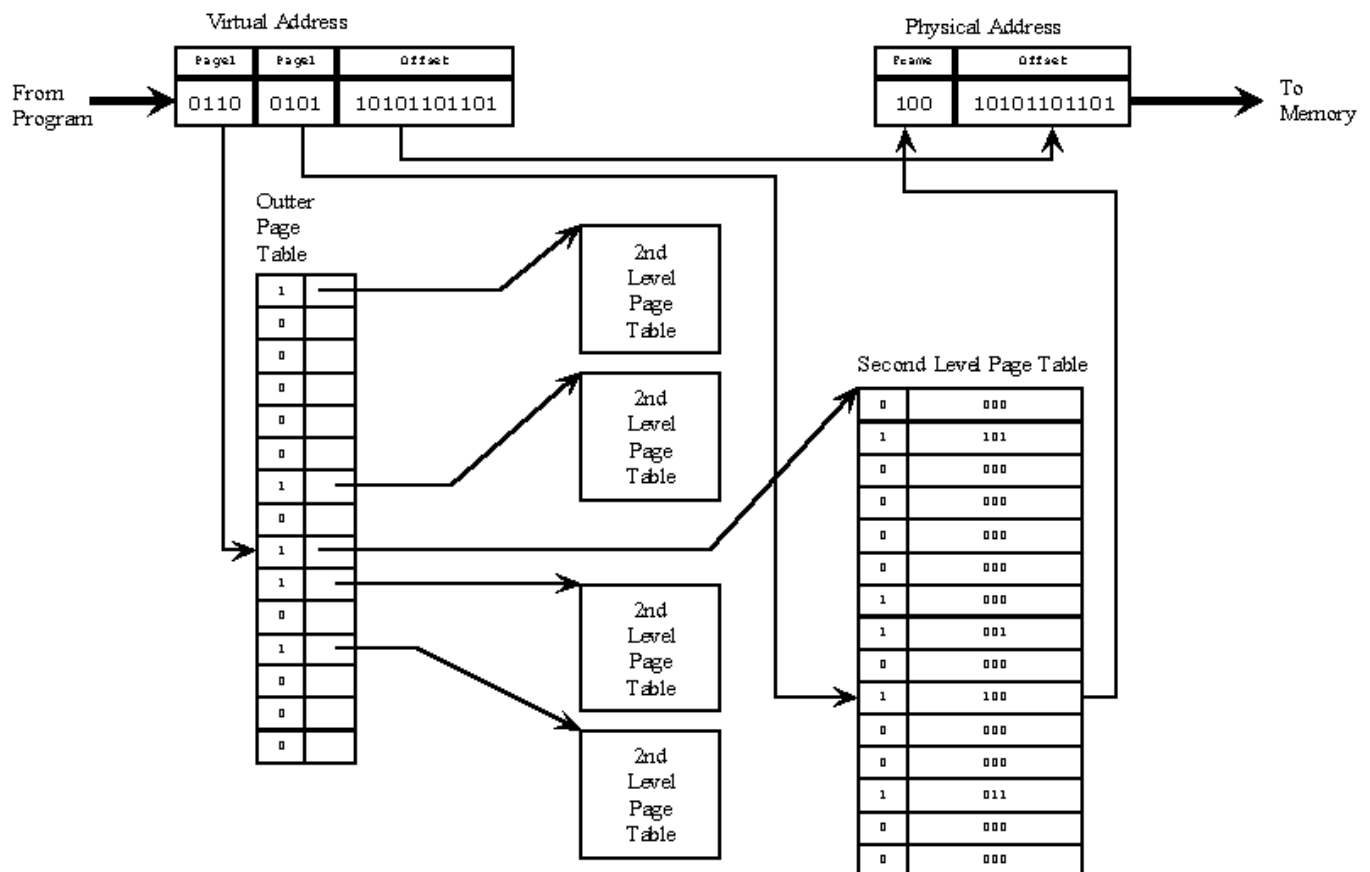
Page Table

Page Tables sind ein wichtiger Bestandteil des Paging, bei dem Prozesse in Pages aufgeteilt werden und in Frames auf dem physischen Speicher geladen werden. Page Tables innerhalb von Prozessen dienen dazu, zu wissen welche Page sich in welchem Frame befindet, und sind die Datenstruktur, die für das Umrechnen von virtuellen zu physischen Adressen verwendet wird. Virtuelle Adressen

werden von Prozessen verwendet und sind eine Abstraktion vom physischen Speicher. Der Prozess kann nur auf seinen eigenen virtuellen Speicher schreiben und lesen. Das Umrechnen der virtuellen Adresse eines Prozesses wird heute üblicherweise von der MMU übernommen und ist für den Prozess gar nicht einsehbar. Die Seitentabellen beinhalten dabei das Mapping zwischen der virtuellen und der physischen Adresse.

Multilevel Page Table

Bei großen Prozessen benötigt die Seitentabelle selbst mehrere Seiten. Deswegen verwendet man dann MPTs.



Paging Speicherschutz

Durch Paging ist Speicherschutz gewährleistet, da jeder Prozess einen eigenen virtuellen Adressraum (Speicherkontext) zugeordnet bekommt und somit von den anderen Prozessen abgeschirmt ist.

logische Struktur des I/O-Interface, einheitliches Programmierinterface

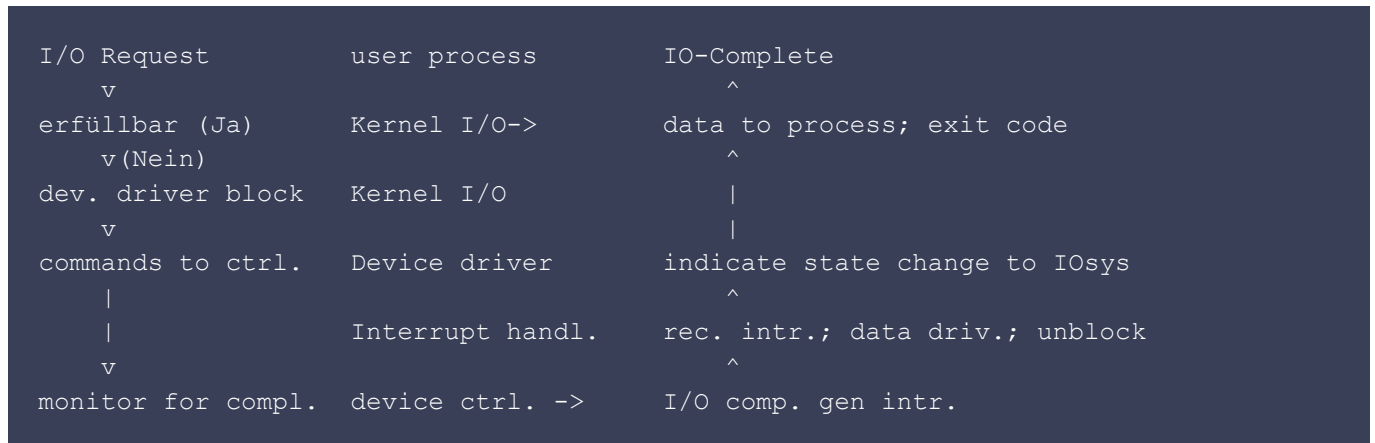
Man benötigt das für die Ansteuerung von Peripheriegeräten. Man hat dabei eine Schichtung:

- Logical I/O: logische Bedienung (read,write,close)
- Device I/O: Maschinenkommandos (I/O), Buffering
- Scheduling und Control: Queueing, Verwaltung, Interrupts, Status

Synchronous vs. Unsynchronous I/O

sehr ähnlich zu blocking vs. nonblocking

Synchroner I/O Request



Disk Scheduling

Disk Scheduling ist dafür zuständig, die Zugriffszeiten auf extern liegende Daten (Disk I/O) zu minimieren. Es ist wichtig weil:

- Mehrere I/O Requests könnten gleichzeitig ankommen, aber nur einer kann abgearbeitet werden. Die anderen Requests warten dann in einer Queue und müssen gescheduled werden.
- Die Requests könnten weit auseinander liegen und somit längere Armebewegungen benötigen.
- Externer Speicher (vor allem Harddisks) sind mitunter die langsamsten Komponenten eines Rechnersystems und müssen dementsprechend sehr gut optimiert werden.

Die dabei zu beachtenden Zeiten sind die Seek Time, der rotational Delay und die Transfer Time. Die Zeit die durch D.S. minimiert wird ist die Seek Time. Drei Strategien dafür sind:

- FIFO: selbsterklärend
- LIFO: dito
- SSTF: Shortest Service Time First

Scheduling-Terms

- Long-term Scheduling
 - Kreierung von Prozessen
 - bestimmt Grad der Parallelität
 - Mix von CPU- und I/O-intensiven Prozessen
- Medium-term Scheduling
 - Ein- und Auslagern von Prozessen (Memory Management)
- Short-term Scheduling
 - bestimmt nächsten Prozess zur Ausführung

Scheduling Optimierungsziele

- Durchsatz
 - zwecks Performance, dass so viele Prozesse wie möglich durchgebracht werden
- Prozessorauslastung
 - dass die CPU nicht Zeit verschwendet mit I/O lastigen Prozessen
- Fairness
 - dass kurze oder lange Prozesse oder I/O lastige Prozesse nicht benachteiligt werden
- Response Time
- Einhalten von zeitlichen Vorgaben

Swapping

Swapping ist innerhalb der Speicherhierarchie das Auslagern von Daten einer höheren in eine tiefere Ebene oder das entgegengesetzte Laden. Durch Swapping sollen die Vorteile höherer Ebenen, also höhere Geschwindigkeit, und tieferer Ebenen, nämlich höhere Kapazität und kostengünstigerer Speicher, quasi gleichzeitig nutzbar gemacht werden. Das wird erkaufte durch den Zeitbedarf des Swappens. Üblicherweise bezieht der Begriff sich auf die Datenverschiebung zwischen Arbeitsspeicher und Festplatte.

Thrashing

Durch häufige Page Faults verbringt der Prozessor sehr viel Zeit mit dem Laden vom Sekundärspeicher

--> drastischer Einbruch der Effektivität (z.B., Resident Set eines Prozesses zu klein)

Working-Set Strategie

= Variable Allokation von Frames für einen Prozess basierend auf Lokalitätsannahme

- Working Set eines Prozesses wächst beim Start schnell an,
- stabilisiert sich während der weiteren Prozessausführung (Lokalitätsprinzip),
- wächst wieder, wenn sich die Lokalität verschiebt, ...

Ermittlung der Größe des Resident Set:

- Beobachtung des Working Set der Prozesse
- Periodisches Löschen der Seiten, die sich nicht im Working Set befinden
- Wenn Resident Set $\cap$ Working Set
 - allokieren fehlende Frames
 - sind nicht genügend Frames verfügbar, suspendiere den Prozess und probiere Allokation später nochmals

Zugriffszeiten Festplatte

- Seek Time: Zeit um Lesearm zu bewegen
- Rotational Delay: ca. halbe Umdrehungszeit der Disk, Zeit bis Sektor erreicht. ($1/(2 \cdot r)$)
- Transfer Time: Übertragungszeit ($b/(r \cdot N)$) (N..Bytes pro Spur, b..zu Übertragende Bytes)

Reduktion durch :

- Priority
- First-In, First-Out (FIFO)
- Last-In, First-Out (LIFO)
- Shortest Service Time First (SSTF)
- SCAN (Elevator Alg.)
- C-Scan
- N-step-SCAN und FSCAN
- RAID

Bestandteile eine File Systems

Die Disk ist partitioniert in verschiedene Teile (für gewöhnlich, es kann auch eine Einzelpartition geben). Die Partitionen sind in einer Partition Table angeführt welche (zusammen mit dem Boot Code) beim Master Boot Record ist. Jede Partition hat ein eigenes File-System, welches (in Unix) so aufgeteilt ist:

- Boot Block
- Super Block
- free Block management
- i-nodes
- root directory
- files & directories

Beim Hochfahren wird der MBR angesteuert (mit oder ohne Boot Menu) welches in der Partition Table eine Bootbare Partition selektiert. In der Partition wird der Boot Block zum einrichten des Filesystems in die Laufzeit durchlaufen. Der Superblock beschreibt den Zustand des Filesystems

Blöcke in Dateisystemen

- Contiguous Allocation
 - aufeinanderfolgende Blöcke verwendet
 - Vorteil: keine Fragmentierung
 - Nachteil: Problem, wenn File vergrößert wird
- Chained Allocation
 - Jeder Block hat einen Pointer auf den nächsten Block
 - Vorteil: keine Fragmentierung
 - Nachteil: Keine Locality in den Blöcken, schlechter Random Access
- Indexed Allocation
 - Tabelle die für jedes File die Blöcke kennt

- Vorteil: direkter Zugriff auf Daten, Blöcke für Userdaten
- Nachteil: FAT braucht viel Speicher
- i-nodes (unklar ob hier gefragt, 4 Blockallozierungen...)
 - Kontrollstruktur, die alle Informationen über ein File speichert
 - Nachteil: Anzahl der Blockreferenzen limitiert

Clock Policy

Ringpuffer mit Frames, die für Austausch in Frage kommen + Positionszeiger. Kaum schlechter als LRU

- nach Page Fault: Zeiger wird auf den folgenden Frame im Ringpuffer gesetzt
- *Use Bit* pro Frame, wird auf 1 gesetzt wenn Seite geladen oder referenziert wird
- bei Page Fault:
 - Suche erste Seite ab Zeigerposition mit Use Bit = 0
 - Setze dabei Use Bits mit Wert 1 von 1 auf 0

i-nodes

Ein Inode (= Index Node) ist die grundlegende Datenstruktur zur Verwaltung von Dateisystemen mit unixartigen Betriebssystemen. Jeder Inode wird innerhalb einer Partition eindeutig durch seine Inode-Nummer identifiziert. Jeder Namenseintrag in einem Verzeichnis verweist auf genau einen Inode. Dieser enthält die Metadaten der Datei und verweist auf die Daten der Datei beziehungsweise die Dateiliste des Verzeichnisses.

Blocking vs. Non-Blocking I/O

- Blocking I/O Prozess wird von Zustand Running in die Blocked Queue gestellt und blockiert. Er wartet quasi bis die I/O wieder freigegeben wird und ist auf die I/O angewiesen.
- Non-Blocking I/O I/O Operation wird sofort fertiggestellt, kein Blockieren, Return Status liefert Feedback über Operation (z.B. Anzahl der übertragenen Bytes). Hierbei wird zu einem späteren Zeitpunkt wieder ein Zugriff auf die I/O versucht.

Buffering

= Zwischenspeicher für Daten bei I/O-Transfer

- Reduktion des Overheads: Zusammenfassung von low-level I/O-Operationen
- Entkopplung Prozess I/O – BS I/O
- BS: Entkopplung I/O – Swapping
- Abfangen von Spitzenlastfällen (mehrere Puffer)

Achtung: Langzeit-Performance nie besser als Device Performance

Kosten: Speicher, Puffermanagement (Zuordnung: Puffer – Prozesse)

File Allocation Table

Ist notwendig für die Strategie der Indexed Allocation. Die File Allocation Table befindet sich im Memory und nicht im Storage und zeigt für jedes File auf welchem Speicherblock sich dieses befindet. Der Vorteil eines solchen Systems ist dass direkter Zugriff sehr gut unterstützt wird. Der Nachteil ist klar ersichtlich -> die FAT muss in den Arbeitsspeicher geladen werden.

Access Control List vs. Capabilities

- Bei Access Control Lists werden die Zugriffsrechte bei den Objekten gespeichert (entspricht den Spalten der ACM). Dabei kann man bei den Objekten die Rechte nach den Gruppen Aufteilen und auch leicht per Objekt Rechte ändern.
- Capability Lists geben Subjekten (also Prozessen) "Tickets" welche Zugriff auf bestimmte Objekte ermöglichen. Das eignet sich gut wenn man Capabilities vererben oder weitergeben will. Dafür müssen diese Tickets aber sehr sicher sein und verschlüsselt werden.

Security Threats

Die drei Kategorien für Gefahren sind:

- Denial of Service: der Zugriff auf ein Service wird gekappt (permanent oder kurzzeitig) - das ist eine Verletzung der Availability
- Exposure/Interception: beschreibt passive Attacken, bei denen "gelauscht" wird ohne Zugriffsberechtigung - das ist eine Verletzung der Confidentiality
- Modification/Fabrication: eine aktive Attacke, bei der gezielt falsche Informationen fabriziert und gespeichert werden - das ist eine Verletzung der Integrity

Bell LaPadula

Das Konzept von Bell LaPadula sagt aus: "No read up, no write down". Es muss klar definierte Security Classifications geben, welche für jedes Subject (S(S)) und jedes Object (S(O)) definiert sind.

- Für Lesezugriffe gilt: $S(S) \geq S(O)$
- Für Appenzugriffe gilt: $S(S) \leq S(O)$
- Für LeseSchreibzugriffe gilt: $S(S) = S(O)$

Designprinzipien für sichere Systeme

- Open Design: nicht "Security by Obscurity"
- Default-Einstellung: keine Berechtigung
- Least Privilege
- Economy of Mechanisms: Fehlervermeidung durch Einfachheit der Sicherheitsmechanismen, Implementierung auf möglichst niedriger Ebene
- Acceptability: akzeptierbare Mechanismen
- Überprüfung der gegenwärtigen Berechtigung

- Complete Mediation: Kontrolle aller Zugriffe auf Ressourcen (auch Ausnahmesituationen)

Naming Strategien in Verteilten Datensystemen (~ ist das im Stoff?)

TCP/IP Layer (~ ist das im Stoff?)

DNS (~ ist das im Stoff?)